Software Engineering Lab (CSE3132)

Experiment No. 6

Title: Introduction to DevOps, Cloud Computing and Virtualization, Migration to DevOps, DevOps Tools

Session Outcome: Use of DevOps tools for software engineering practice

Project Considered: Student Academic Project Management System

1. Aim

To understand what DevOps is and to use DevOps tools (Git, GitHub Actions, Docker, pytest and flake8) on our project, so that the code is saved safely, tested automatically and packed so that it runs the same on any computer.

2. Theory (in simple words)

2.1 What is DevOps?

Normally, **developers** write the code and a different group, the **operations** team, runs it on servers. These two groups often blame each other when something breaks. **DevOps** means both groups work together and use tools to do the boring work (building, testing, deploying) **automatically**. The result is that new features reach the users faster and with fewer mistakes.

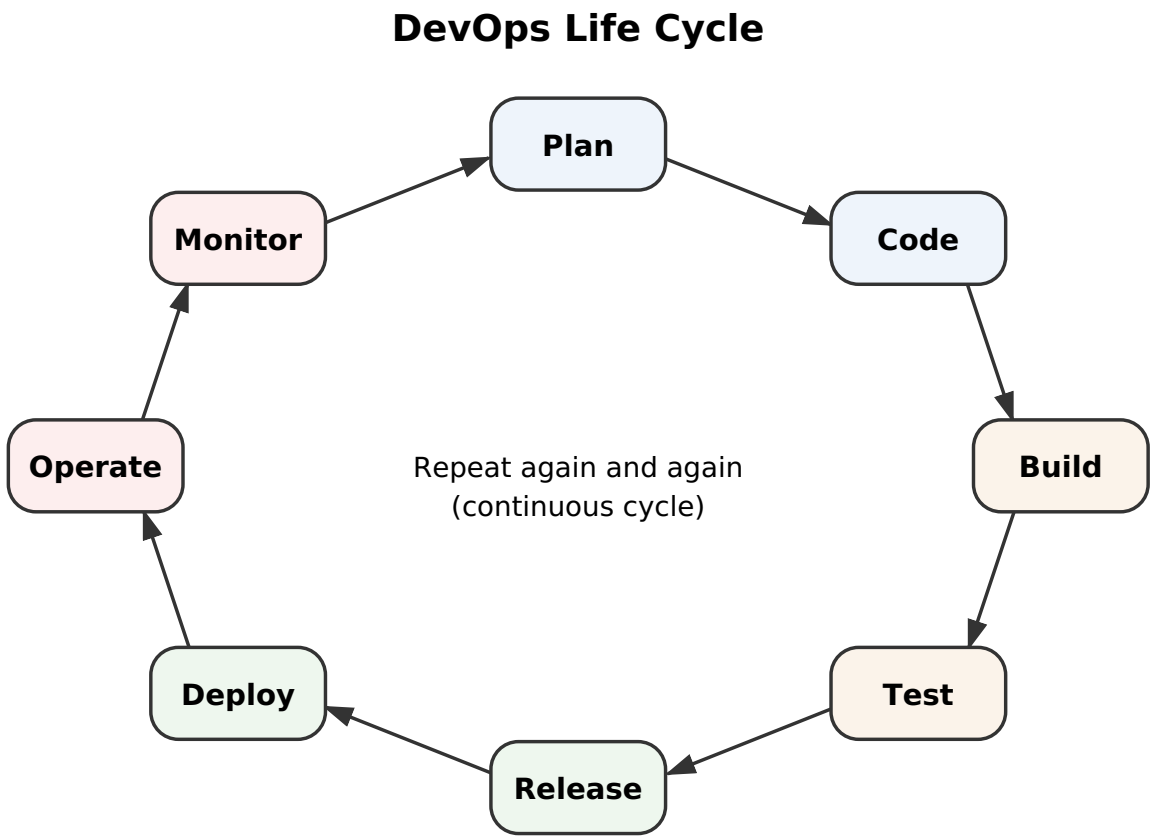


Fig. 1 – DevOps life cycle (it keeps repeating)

2.2 What is Cloud Computing?

Cloud computing means using someone else's computers over the internet instead of buying our own. We pay only for what we use. Examples are AWS, Google Cloud and Microsoft Azure. For our project, the app could be placed on a cloud server so students and faculty can open it from anywhere (this is listed in our future scope).

2.3 What is Virtualization and a Container?

Virtualization means running many "virtual computers" on one real computer. A **virtual machine (VM)** has a full operating system inside it, so it is heavy. A **container** is lighter – it packs only our app and the things it needs (Python, Flask, etc.) and shares the operating system of the host. **Docker** is the most popular container tool. The famous problem "it works on my laptop but not on yours" is solved by containers.

Virtual Machine	Container (Docker)
Has a full operating system	Shares the host operating system
Heavy (GBs), slow to start	Light (MBs), starts in seconds
Strong separation	Lighter separation, but enough for most apps

2.4 Migration to DevOps – before and after

Before (our old way)	After (DevOps way)
Code shared through WhatsApp / pen drive	Code kept in Git and GitHub with full history
We test by clicking around manually	pytest tests run automatically
Style mistakes found late	flake8 checks style on every push
"Works only on my laptop"	Docker container runs the same everywhere
Someone must remember to run everything	GitHub Actions does it automatically on pushes to main and pull requests to main.

2.5 DevOps tools used in this experiment

Tool	What it does	Used for in our project
Git	Version control (saves every change)	Keeping history of our code
GitHub	Website to store Git projects online	Sharing code between both team members
pytest	Runs automatic tests	Testing login, proposal and review routes
flake8	Checks code style	Finding style mistakes early
Docker	Packs the app in a container	Running the app on any computer
GitHub Actions	Runs steps automatically (CI/CD)	Auto build and test on every push

3. Step 1 – Version Control with Git

First, a Git repository was created for the project. The work was done in small steps and each step was saved as a **commit** with a clear message. The DevOps files were developed on a separate **branch** called `devops-setup` so that the main code stays safe, and then **merged** back into `main`.

```
$ git init
$ git add db.py app.py .gitignore
$ git commit -m "Add database setup and Flask app"
$ git checkout -b devops-setup        (make a new branch)
$ git commit ...                      (Dockerfile and CI file added here)
$ git checkout main
$ git merge --no-ff devops-setup      (join the branch back to main)
```

Output of `git log --oneline --graph` (real output):

```
*   b2aae81 Merge devops-setup into main
|\
| * 33945d8 Add GitHub Actions CI workflow
| * a9a5bcf Add Dockerfile for containerization
|/
* 6763f86 Add requirements and pytest tests
* 0181341 Add database setup and Flask app
```

Files tracked by Git (`git ls-files`):

```
.dockerignore
.github/workflows/ci.yml
.gitignore
Dockerfile
app.py
db.py
requirements.txt
test_app.py
```

A `.gitignore` file was added so that temporary files like `__pycache__` and the database file `spms.db` are not saved in Git.

4. Step 2 – Automatic Tests with pytest

Seven small tests were written. Every test uses a fresh temporary database, so one test cannot spoil another. These tests are what the CI pipeline runs later.

`test_app.py`

```
"""
test_app.py
Automated tests for the SPMS app. These are run by pytest, both on my laptop
and by GitHub Actions every time we push code.

Each test uses a fresh temporary database so tests don't affect each other.
"""

import pytest
import app as flaskapp
import db


@pytest.fixture
def client(tmp_path, monkeypatch):
    # point the app to a temp database file instead of the real spms.db
    monkeypatch.setattr(db, "DB_NAME", str(tmp_path / "test.db"))
    db.init_db()
    return flaskapp.app.test_client()


def add_user(client, name, email, role):
    r = client.post("/register", json={
        "name": name, "email": email, "password": "pass123", "role": role
    })
    return r.get_json()["user_id"]


def test_register_and_login(client):
    add_user(client, "Saksham", "s@test.com", "student")
    r = client.post("/login", json={"email": "s@test.com", "password": "pass123"})
    assert r.status_code == 200
    assert r.get_json()["role"] == "student"


def test_login_wrong_password(client):
    add_user(client, "Saksham", "s@test.com", "student")
    r = client.post("/login", json={"email": "s@test.com", "password": "wrong"})
    assert r.status_code == 401


def test_register_missing_field(client):
    r = client.post("/register", json={"name": "No Email", "role": "student"})
    assert r.status_code == 400
```

```python
def test_register_invalid_role(client):
    r = client.post("/register", json={
        "name": "X", "email": "x@test.com", "password": "p", "role": "hacker"
    })
    assert r.status_code == 400


def test_proposal_flow(client):
    fac = add_user(client, "Faculty", "f@test.com", "faculty")
    stu = add_user(client, "Student", "s@test.com", "student")

    t = client.post("/team/create", json={"team_name": "T1", "member_ids": [stu]})
    team_id = t.get_json()["team_id"]

    p = client.post("/proposal/submit", json={
        "title": "My Project", "description": "demo",
        "team_id": team_id, "faculty_id": fac
    })
    assert p.status_code == 201
    assert p.get_json()["status"] == "Pending"
    pid = p.get_json()["project_id"]

    r = client.post(f"/proposal/{pid}/review", json={"decision": "approve"})
    assert r.get_json()["status"] == "Approved"

    g = client.get(f"/proposal/{pid}")
    assert g.get_json()["status"] == "Approved"


def test_review_invalid_decision(client):
    r = client.post("/proposal/1/review", json={"decision": "maybe"})
    assert r.status_code == 400


def test_proposal_not_found(client):
    r = client.get("/proposal/999")
    assert r.status_code == 404
```

5. Step 3 – Containerization with Docker

A `Dockerfile` is a recipe that tells Docker how to build the container. Ours has these simple steps:

`Dockerfile`

```
# Dockerfile for the Student Academic Project Management System

# start from a small official python image
FROM python:3.12-slim

# folder inside the container where our code will live
WORKDIR /app

# copy requirements first so docker can cache the install step
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# now copy the rest of the project
COPY . .

# flask runs on port 5000
EXPOSE 5000

# create the database tables, then start the app using gunicorn
CMD ["sh", "-c", "python db.py && gunicorn -b 0.0.0.0:5000 app:app"]
```

Line	Meaning
FROM python:3.12-slim	Start from a small ready-made Python image
WORKDIR /app	Work inside the folder /app in the container
COPY requirements.txt + RUN pip install	Install the libraries our app needs
COPY . .	Copy our project code into the container
EXPOSE 5000	The app listens on port 5000
CMD ...	Create the database tables and start the app with gunicorn (a proper web server)

A `.dockerignore` file keeps unwanted files (like `.git` and `spms.db`) out of the image.

Commands to build and run the container:

```
$ docker build -t spms-app .
$ docker run -p 5000:5000 spms-app
(now open http://localhost:5000 in the browser)
```

> **Note:** Docker was not installed on the computer used to prepare this report, so the two commands above were not run here and no output is shown. They should be run on a computer with Docker installed, and a screenshot of the result can be added at this place.

6. Step 4 – Continuous Integration with GitHub Actions

CI (Continuous Integration) means that every time someone pushes code, a robot automatically builds and tests it. The workflow file below tells GitHub what to do. It is kept inside the folder `.github/workflows/`.

`.github/workflows/ci.yml`

```
# CI pipeline - runs automatically on every push / pull request to main
name: SPMS CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Get the code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: pip install -r requirements.txt

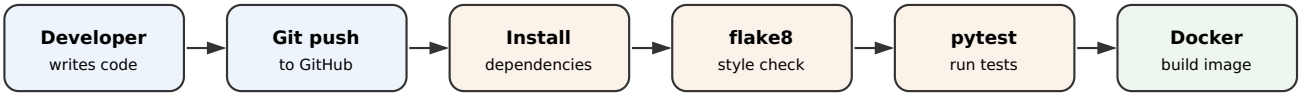
      - name: Check code style with flake8
        run: flake8 . --max-line-length=110 --exclude=.git,__pycache__

      - name: Run tests with pytest
```

```
        run: pytest -v

      - name: Build Docker image
        run: docker build -t spms-app .
```

CI Pipeline of our project (GitHub Actions)



If any step fails, the pipeline stops and shows a red cross on GitHub. If all steps pass, we get a green tick.

Fig. 2 – Steps of our CI pipeline

Running the same steps on the local computer

Since the workflow is only started by GitHub after a push, the same style-check and test steps were run by hand on the local computer to see what the pipeline will do. The file was also checked to be valid YAML. Real output:

```
$ pip install -r requirements.txt   (done earlier)
$ flake8 . --max-line-length=110 --exclude=.git,__pycache__
(no output = no style problems)

$ pytest -v
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: spms_app
plugins: cov-7.1.0
collecting ... collected 7 items

test_app.py::test_register_and_login PASSED                              [ 14%]
test_app.py::test_login_wrong_password PASSED                            [ 28%]
test_app.py::test_register_missing_field PASSED                          [ 42%]
test_app.py::test_register_invalid_role PASSED                           [ 57%]
test_app.py::test_proposal_flow PASSED                                   [ 71%]
test_app.py::test_review_invalid_decision PASSED                         [ 85%]
test_app.py::test_proposal_not_found PASSED                              [100%]

============================== 7 passed in 0.76s ===============================
```

7. Result / Conclusion

We learned the basic idea of DevOps, cloud and containers, and applied DevOps tools on our project. The code is now in a Git repository with clear commits, a branch and a merge. Seven automatic tests pass, flake8 finds no style problems, a Dockerfile is ready to pack the app, and a GitHub Actions file will run style check, tests and Docker build automatically on every push. The Docker build and the actual GitHub run were not executed while preparing this report; only the local test and style-check steps were run.